

g86mvkage

May 25, 2026

```
[1]: import tqdm as notebook_tqdm
```

```
[2]: from sentence_transformers import SentenceTransformer
```

```
C:\Users\Admin\anaconda3\envs\env_ter\Lib\site-packages\tqdm\auto.py:21:  
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See  
https://ipywidgets.readthedocs.io/en/stable/user_install.html  
from .autonotebook import tqdm as notebook_tqdm
```

```
[3]: from sklearn.metrics.pairwise import cosine_similarity
```

```
[4]: # Fichiers utilisés  
OFFRE_PATH = "Offre.txt"  
CV_PATHS = {  
    "CV1 (Sophie Marchand)": "20CVs/cv_01.txt",  
    "CV2 (Rayan Kowalski)": "20CVs/cv_02.txt",  
    "CV3 (James Fletcher)": "20CVs/cv_03.txt",  
    "CV4 (Camille Nguyen)": "20CVs/cv_04.txt",  
    "CV5 (Tariq Mansouri)": "20CVs/cv_05.txt",  
    "CV6 (Maxime Berthelot)": "20CVs/cv_06.txt",  
    "CV7 (Lorenzo Ricci)": "20CVs/cv_07.txt",  
    "CV8 (Fatou Diallo)": "20CVs/cv_08.txt",  
    "CV9 (Théo Blanchard)": "20CVs/cv_09.txt",  
    "CV10 (Amandine Lefebvre)": "20CVs/cv_10.txt",  
    "CV11 (Nina Castillo)": "20CVs/cv_11.txt",  
    "CV12 (Pierre Gautier)": "20CVs/cv_12.txt",  
    "CV13 (Sébastien Morin)": "20CVs/cv_13.txt",  
    "CV14 (Yuna Park)": "20CVs/cv_14.txt",  
    "CV15 (Mehdi Tazi)": "20CVs/cv_15.txt",  
    "CV16 (Lucie Renard)": "20CVs/cv_16.txt",  
    "CV17 (Guillaume Faure)": "20CVs/cv_17.txt",  
    "CV18 (Alexia Mercier)": "20CVs/cv_18.txt",  
    "CV19 (Raphaël Cordier)": "20CVs/cv_19.txt",  
    "CV20 (Isabelle Dumont)": "20CVs/cv_20.txt",  
}
```

```
[5]: # fonction pour lire un fichier
```

```
def load_file(file_path):  
    with open(file_path, "r", encoding="utf-8") as f:  
        file = f.read()  
    return file
```

```
[6]: # lire le fichier de l'offre
```

```
offre = load_file(OFFRE_PATH)
```

```
[7]: offre
```

```
[7]: "Offre d'emploi : Ingénieur NLP / Développeur IA\nEntreprise : DataRecruit  
Solutions - Montpellier, France\n\nNous recherchons un développeur passionné par  
le traitement automatique du langage naturel (NLP).\nVous travaillerez sur des  
pipelines d'analyse de texte utilisant des embeddings et la similarité cosinus  
pour comparer des documents.\nUne maîtrise solide de Python est indispensable,  
ainsi qu'une expérience avec des bibliothèques comme spaCy, NLTK ou  
HuggingFace.\nVous serez chargé de concevoir et déployer des API robustes avec  
FastAPI pour exposer nos modèles de traitement.\nLa gestion du code source via  
Git et GitHub est obligatoire pour collaborer efficacement au sein de  
l'équipe.\nUne expérience avec des modèles de langage (LLM) et la vectorisation  
de texte est fortement appréciée.\nVous évoluerez dans un environnement agile,  
en lien direct avec les équipes produit et data.\nPoste en CDI, rémunération  
selon profil. Candidature à envoyer par e-mail avec votre CV et lettre de  
motivation.\n"
```

```
[ ]:
```

```
[8]: # charger le model
```

```
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights:

100% | 199/199

[00:00<00:00, 1553.63it/s]

BertModel LOAD REPORT from: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

Key	Status	
embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

1 encoder l'offre

```
[9]: embedding_offre = model.encode([offre])
```

```
[10]: embedding_offre.shape
```

```
[10]: (1, 384)
```

```
[ ]:
```

2 Quelques fonctions utiles

```
[11]: def pretraiter_cv(texte):
    texte = re.sub(r'\\[\\[.*?\\]\\]', r'\\1', texte)
    texte = re.sub(r'>>>|>>|===|---|\\*\\*\\*|\\\\\\\\', '\\n', texte)
    texte = re.sub(r'\\r\\n', '\\n', texte)
    texte = re.sub(r'\\n{3,}', '\\n\\n', texte)
    texte = re.sub(r'[• → ]', ' ', texte)
    texte = re.sub(r' +', ' ', texte)
    return texte.strip()
```

```
[12]: def extract_results(embedding_offre, cvs_chunk_strategy):
    resultats = []

    for nom, blocs in cvs_chunk_strategy.items():
        # Encoder tous les blocs du CV d'un coup
        embeddings_blocs = model.encode(blocs)

        #calculer la similarité cosinus entre l'offre et chaque bloc
        scores_blocs = cosine_similarity(embedding_offre, embeddings_blocs)[0]
        #print(scores_blocs)

        meilleur_idx = scores_blocs.argmax()
        score_cv = scores_blocs[meilleur_idx]
        meilleur_bloc = blocs[meilleur_idx]

        resultats.append({
            "nom": nom,
            "score": float(score_cv),
            "meilleur_idx": meilleur_idx,
            "meilleur_bloc": meilleur_bloc,
            "nb_blocs": len(blocs),
            "scores_blocs": scores_blocs,
        })

    return resultats
```

```
[13]: # 6. Classement
def classement(resultats, strategy_name):
    resultats.sort(key=lambda x: x["score"], reverse=True)

    score = round(resultats[0]['score'], 4)
    nom = resultats[0]['nom']

    print("=" * 60)
    print(f"          CLASSEMENT DES CANDIDATS (avec chunking :↳
↳{strategy_name})")
    print("=" * 60)

    for i, r in enumerate(resultats):
        barre = " " * int(r["score"] * 30)
        print(f"\n{r['nom']}")
        print(f"    Score : {r['score']:.4f}  {barre}")
        print(f"    Bloc décisif : n°{r['meilleur_idx'] + 1} / {r['nb_blocs']}")

    print("\n" + "=" * 60)

    # 7. Détail du meilleur candidat

    gagnant = resultats[0]
    print(f"\n MEILLEUR CANDIDAT : {gagnant['nom']}")
    print(f" Score final : {gagnant['score']:.4f}\n")
    print(" Tous les scores par bloc :")
    for i, s in enumerate(gagnant["scores_blocs"]):
        marqueur = " ← meilleur" if i == gagnant["meilleur_idx"] else ""
        print(f"    Bloc {i} : {s:.4f}{marqueur}")

    print(f"\n Contenu du bloc décisif (bloc {gagnant['meilleur_idx']}) :")
    print("-" * 60)
    print(gagnant["meilleur_bloc"])
    print("-" * 60)

    return nom, score # retourner le meilleur score
```

```
[ ]:
```

3 Stratégie 1 : découpage par sauts de lignes

```
[14]: import re
import numpy as np
```

```
[15]: def chunk_by_newlines(texte):
    """
```

```

Découpage par double saut de ligne (\n\n)
"""
texte = pretraiter_cv(texte)
blocs = texte.strip().split("\n\n")
blocs = [b.strip() for b in blocs if b.strip()]
return blocs

```

```

[16]: # Juste pour testerµ
      #blocs_cv1 = chunk_by_newlines(load_file("Tests/cv_test_01.txt"))

```

```

[17]: cvs_chunk_by_newlines = {nom: chunk_by_newlines(load_file(chemin)) for nom,
      ↪chemin in CV_PATHS.items()}

```

```

[18]: for i, (nom, blocs) in enumerate(cvs_chunk_by_newlines.items()):
      print(f"{nom} => {len(blocs)}")

```

```

CV1 (Sophie Marchand) => 6
CV2 (Rayan Kowalski) => 1
CV3 (James Fletcher) => 10
CV4 (Camille Nguyen) => 6
CV5 (Tariq Mansouri) => 5
CV6 (Maxime Berthelot) => 2
CV7 (Lorenzo Ricci) => 12
CV8 (Fatou Diallo) => 1
CV9 (Théo Blanchard) => 4
CV10 (Amandine Lefebvre) => 6
CV11 (Nina Castillo) => 4
CV12 (Pierre Gautier) => 4
CV13 (Sébastien Morin) => 1
CV14 (Yuna Park) => 6
CV15 (Mehdi Tazi) => 1
CV16 (Lucie Renard) => 1
CV17 (Guillaume Faure) => 5
CV18 (Alexia Mercier) => 5
CV19 (Raphaël Cordier) => 7
CV20 (Isabelle Dumont) => 1

```

```

[19]: resultats_chunk_by_newlines = extract_results(embedding_offre,
      ↪cvs_chunk_by_newlines)

```

```

[20]: classement(resultats_chunk_by_newlines, "chunk by newlines")

```

```

=====
          CLASSEMENT DES CANDIDATS (avec chunking : chunk by newlines)
=====

```

```

CV3 (James Fletcher)

```

Score : 0.7255
Bloc décisif : n°5 / 10

CV6 (Maxime Berthelot)
Score : 0.7163
Bloc décisif : n°2 / 2

CV1 (Sophie Marchand)
Score : 0.7158
Bloc décisif : n°3 / 6

CV10 (Amandine Lefebvre)
Score : 0.6966
Bloc décisif : n°3 / 6

CV8 (Fatou Diallo)
Score : 0.6787
Bloc décisif : n°1 / 1

CV11 (Nina Castillo)
Score : 0.6635
Bloc décisif : n°2 / 4

CV4 (Camille Nguyen)
Score : 0.6595
Bloc décisif : n°3 / 6

CV14 (Yuna Park)
Score : 0.6508
Bloc décisif : n°3 / 6

CV15 (Mehdi Tazi)
Score : 0.6461
Bloc décisif : n°1 / 1

CV5 (Tariq Mansouri)
Score : 0.6352
Bloc décisif : n°4 / 5

CV9 (Théo Blanchard)
Score : 0.5952
Bloc décisif : n°4 / 4

CV13 (Sébastien Morin)
Score : 0.5929
Bloc décisif : n°1 / 1

CV7 (Lorenzo Ricci)

Score : 0.5847
Bloc décisif : n°6 / 12

CV16 (Lucie Renard)
Score : 0.5506
Bloc décisif : n°1 / 1

CV2 (Rayan Kowalski)
Score : 0.5426
Bloc décisif : n°1 / 1

CV12 (Pierre Gautier)
Score : 0.5247
Bloc décisif : n°4 / 4

CV17 (Guillaume Faure)
Score : 0.4987
Bloc décisif : n°3 / 5

CV20 (Isabelle Dumont)
Score : 0.4949
Bloc décisif : n°1 / 1

CV18 (Alexia Mercier)
Score : 0.4940
Bloc décisif : n°3 / 5

CV19 (Raphaël Cordier)
Score : 0.4876
Bloc décisif : n°5 / 7

=====

MEILLEUR CANDIDAT : CV3 (James Fletcher)
Score final : 0.7255

Tous les scores par bloc :
Bloc 0 : 0.2772
Bloc 1 : 0.2854
Bloc 2 : 0.5294
Bloc 3 : 0.2995
Bloc 4 : 0.7255 ← meilleur
Bloc 5 : 0.4303
Bloc 6 : 0.2680
Bloc 7 : 0.6045
Bloc 8 : 0.4498
Bloc 9 : 0.2178

Contenu du bloc décisif (bloc 4) :

Research Scientist, Naver Labs Europe Grenoble, 2020 to present. Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023.

[20]: ('CV3 (James Fletcher)', 0.7255)

[]:

4 Stratégie 2 : découpage par taille fixe avec overlap

```
[21]: def chunk_by_words(texte, taille=80, overlap=20):  
    """  
    Cette fonction sert à découper un texte en petits blocs (chunks) de mots.  
    - taille=80 → chaque bloc contient 80 mots  
    - overlap=20 → les blocs se chevauchent de 20 mots  
    """  
    texte = pretraiter_cv(texte)  
    mots = texte.split() # découper le texte en une liste de mots  
    blocs = [] # liste qui va contenir tous les morceaux de texte  
    i = 0  
    while i < len(mots):  
        bloc = " ".join(mots[i:i + taille]) # prend 80 mots a partir de la pos_  
        ↪ "i" et transforme en text ( l'utilité de " ".join )  
        blocs.append(bloc)  
        # on commence pas le prochain bloc a partir de l'index i+taille,  
        # mais plutot on commence quelques mots avant cet index  
        # pk ?? pour réduire le risque de couper une phrase au milieu, pour_  
        ↪ qu'elle ait pas un  
        # sens qui veut rien dire ==> pour augmenter la chance d'avoir des_  
        ↪ phrases qui ont du sens  
        i += taille - overlap  
    return blocs
```

```
[22]: cvs_chunk_by_words = {nom: chunk_by_words(load_file(chemin)) for nom, chemin in_  
    ↪ CV_PATHS.items()}
```

```
[23]: resultats_chunk_by_words = extract_results(embedding_offre, cvs_chunk_by_words)
```

```
[24]: classement(resultats_chunk_by_words, "chunk by words")
```

=====

CLASSEMENT DES CANDIDATS (avec chunking : chunk by words)

=====

CV3 (James Fletcher)
Score : 0.7233
Bloc décisif : n°2 / 3

CV6 (Maxime Berthelot)
Score : 0.7182
Bloc décisif : n°2 / 3

CV7 (Lorenzo Ricci)
Score : 0.7089
Bloc décisif : n°2 / 3

CV10 (Amandine Lefebvre)
Score : 0.7071
Bloc décisif : n°2 / 3

CV12 (Pierre Gautier)
Score : 0.6907
Bloc décisif : n°2 / 2

CV4 (Camille Nguyen)
Score : 0.6810
Bloc décisif : n°2 / 4

CV9 (Théo Blanchard)
Score : 0.6803
Bloc décisif : n°3 / 3

CV8 (Fatou Diallo)
Score : 0.6787
Bloc décisif : n°1 / 2

CV1 (Sophie Marchand)
Score : 0.6681
Bloc décisif : n°2 / 3

CV14 (Yuna Park)
Score : 0.6525
Bloc décisif : n°1 / 3

CV5 (Tariq Mansouri)
Score : 0.6497
Bloc décisif : n°2 / 3

CV15 (Mehdi Tazi)
Score : 0.6461
Bloc décisif : n°1 / 2

CV11 (Nina Castillo)
Score : 0.6446
Bloc décisif : n°2 / 3

CV2 (Rayan Kowalski)
Score : 0.6165
Bloc décisif : n°2 / 3

CV16 (Lucie Renard)
Score : 0.6093
Bloc décisif : n°2 / 2

CV13 (Sébastien Morin)
Score : 0.5935
Bloc décisif : n°1 / 2

CV17 (Guillaume Faure)
Score : 0.5855
Bloc décisif : n°2 / 2

CV20 (Isabelle Dumont)
Score : 0.5301
Bloc décisif : n°2 / 2

CV19 (Raphaël Cordier)
Score : 0.5214
Bloc décisif : n°2 / 2

CV18 (Alexia Mercier)
Score : 0.4861
Bloc décisif : n°2 / 2

=====

MEILLEUR CANDIDAT : CV3 (James Fletcher)
Score final : 0.7233

Tous les scores par bloc :
Bloc 0 : 0.5991
Bloc 1 : 0.7233 ← meilleur
Bloc 2 : 0.4465

Contenu du bloc décisif (bloc 1) :

for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023. Research Intern, Google Brain London, 2019. Fine-tuning BERT models for question answering. Experimenting with RAG architectures. SKILLS Python expert, HuggingFace, sentence-transformers, spaCy, PyTorch, FAISS, cosine similarity, embeddings, LLMs, RAG, FastAPI, Docker, Git, GitHub Actions. LANGUAGES English native, French C1, German

[24]: ('CV3 (James Fletcher)', 0.7233)

[]:

5 Stratégie 3 : découpage par phrases

[25]: *#### On coupe à chaque fin de phrase (., !, ?), puis on regroupe les phrases jusqu'à une taille max.*

```
[26]: def chunk_by_sentences(texte, max_mots=100):
    texte = pretraiter_cv(texte)
    phrases = re.split(r'(?<=[.!?])\s+', texte)

    bloc_courant = []
    cpt_mots = 0
    blocs = []

    for phrase in phrases:
        n = len(phrase.split())
        if cpt_mots + n > max_mots and bloc_courant:
            blocs.append(" ".join(bloc_courant))
            bloc_courant = []
            cpt_mots = 0
        bloc_courant.append(phrase)
        cpt_mots = cpt_mots + n

    if bloc_courant:
        blocs.append(" ".join(bloc_courant))
    return blocs
```

```
[27]: cvs_chunk_by_sentences = {nom: chunk_by_sentences(load_file(chemin)) for nom,
    chemin in CV_PATHS.items()}
```

```
[28]: resultats_chunk_by_sentences = extract_results(embedding_offre,
    cvs_chunk_by_sentences)
```

```
[29]: classement(resultats_chunk_by_sentences, "chunk by sentences")
```

```
=====
          CLASSEMENT DES CANDIDATS (avec chunking : chunk by sentences)
=====

CV4 (Camille Nguyen)
  Score : 0.7012
  Bloc décisif : n°2 / 3

CV14 (Yuna Park)
  Score : 0.6936
  Bloc décisif : n°2 / 2

CV8 (Fatou Diallo)
  Score : 0.6787
  Bloc décisif : n°1 / 1

CV6 (Maxime Berthelot)
  Score : 0.6523
  Bloc décisif : n°2 / 2

CV15 (Mehdi Tazi)
  Score : 0.6461
  Bloc décisif : n°1 / 1

CV9 (Théo Blanchard)
  Score : 0.6450
  Bloc décisif : n°2 / 2

CV11 (Nina Castillo)
  Score : 0.6427
  Bloc décisif : n°1 / 2

CV1 (Sophie Marchand)
  Score : 0.6387
  Bloc décisif : n°1 / 1

CV7 (Lorenzo Ricci)
  Score : 0.6174
  Bloc décisif : n°2 / 2

CV3 (James Fletcher)
  Score : 0.5991
  Bloc décisif : n°1 / 2

CV13 (Sébastien Morin)
```

Score : 0.5929
Bloc décisif : n°1 / 1

CV18 (Alexia Mercier)
Score : 0.5682
Bloc décisif : n°2 / 2

CV16 (Lucie Renard)
Score : 0.5662
Bloc décisif : n°2 / 2

CV2 (Rayan Kowalski)
Score : 0.5426
Bloc décisif : n°1 / 1

CV17 (Guillaume Faure)
Score : 0.5283
Bloc décisif : n°1 / 1

CV10 (Amandine Lefebvre)
Score : 0.5269
Bloc décisif : n°2 / 2

CV12 (Pierre Gautier)
Score : 0.5033
Bloc décisif : n°1 / 2

CV20 (Isabelle Dumont)
Score : 0.4949
Bloc décisif : n°1 / 1

CV5 (Tariq Mansouri)
Score : 0.4832
Bloc décisif : n°1 / 1

CV19 (Raphaël Cordier)
Score : 0.4704
Bloc décisif : n°1 / 1

=====

MEILLEUR CANDIDAT : CV4 (Camille Nguyen)
Score final : 0.7012

Tous les scores par bloc :
Bloc 0 : 0.4807
Bloc 1 : 0.7012 ← meilleur
Bloc 2 : 0.6210

Contenu du bloc décisif (bloc 1) :

Durant mon stage de 5 mois chez Alkemics Lyon en 2024, j'ai développé des pipelines de traitement de texte avec Python et spaCy, calculé des similarités cosinus pour le matching de fiches produits, et déployé une API de démonstration avec FastAPI. J'ai également eu ma première expérience avec HuggingFace et sentence-transformers, et j'ai utilisé Git pour le versioning. En parallèle, j'ai mené un projet académique à l'INSA Lyon en 2023 où j'ai implémenté un système RAG avec HuggingFace et FAISS, et effectué du fine-tuning d'un modèle BERT sur des données françaises.

[29]: ('CV4 (Camille Nguyen)', 0.7012)

[]:

6 Stratégie 4 : chunking sémantique

[30]: *#### On encode chaque phrase, on compare chaque phrase avec la suivante, et on coupe là où la similarité chute - ce qui indique un changement de sujet :*

```
[31]: def chunk_semantic(texte):
    texte = pretraiter_cv(texte)
    phrases = re.split(r'(?<=[.!?\n])\s+', texte.strip())
    phrases = [p.strip() for p in phrases if len(p.strip()) > 10]
    if len(phrases) < 3:
        return phrases
    embeddings = model.encode(phrases)
    sims = [cosine_similarity([embeddings[i-1]], [embeddings[i]])[0][0]
            for i in range(1, len(phrases))]
    seuil = np.mean(sims) - np.std(sims)
    blocs, bloc_courant = [], [phrases[0]]
    for i, s in enumerate(sims):
        if s < seuil:
            blocs.append(" ".join(bloc_courant))
            bloc_courant = []
            bloc_courant.append(phrases[i + 1])
    blocs.append(" ".join(bloc_courant))
    return blocs
```

```
[32]: cvs_chunk_semantic = {nom: chunk_semantic(load_file(chemin)) for nom, chemin in
    CV_PATHS.items()}
```

```
[33]: for i, (nom, chunks) in enumerate(cvs_chunk_semantic.items()):
    print(f"Pour le cv {i+1} ==> {len(chunks)}")
```

```

Pour le cv 1 ==> 2
Pour le cv 2 ==> 1
Pour le cv 3 ==> 4
Pour le cv 4 ==> 3
Pour le cv 5 ==> 2
Pour le cv 6 ==> 1
Pour le cv 7 ==> 3
Pour le cv 8 ==> 1
Pour le cv 9 ==> 3
Pour le cv 10 ==> 3
Pour le cv 11 ==> 3
Pour le cv 12 ==> 2
Pour le cv 13 ==> 1
Pour le cv 14 ==> 3
Pour le cv 15 ==> 1
Pour le cv 16 ==> 2
Pour le cv 17 ==> 2
Pour le cv 18 ==> 3
Pour le cv 19 ==> 4
Pour le cv 20 ==> 1

```

```
[34]: resultats_chunk_semantic = extract_results(embedding_offre, cvs_chunk_semantic)
```

```
[35]: classement(resultats_chunk_semantic, "chunk semantic")
```

```

=====
          CLASSEMENT DES CANDIDATS (avec chunking : chunk semantic)
=====

CV3 (James Fletcher)
  Score : 0.6979
  Bloc décisif : n°3 / 4

CV10 (Amandine Lefebvre)
  Score : 0.6822
  Bloc décisif : n°2 / 3

CV8 (Fatou Diallo)
  Score : 0.6787
  Bloc décisif : n°1 / 1

CV6 (Maxime Berthelot)
  Score : 0.6495
  Bloc décisif : n°1 / 1

CV15 (Mehdi Tazi)
  Score : 0.6461
  Bloc décisif : n°1 / 1

```

CV9 (Théo Blanchard)
Score : 0.6450
Bloc décisif : n°2 / 3

CV11 (Nina Castillo)
Score : 0.6427
Bloc décisif : n°1 / 3

CV1 (Sophie Marchand)
Score : 0.6387
Bloc décisif : n°1 / 2

CV4 (Camille Nguyen)
Score : 0.6378
Bloc décisif : n°2 / 3

CV14 (Yuna Park)
Score : 0.6293
Bloc décisif : n°2 / 3

CV13 (Sébastien Morin)
Score : 0.5929
Bloc décisif : n°1 / 1

CV7 (Lorenzo Ricci)
Score : 0.5847
Bloc décisif : n°2 / 3

CV16 (Lucie Renard)
Score : 0.5506
Bloc décisif : n°1 / 2

CV19 (Raphaël Cordier)
Score : 0.5438
Bloc décisif : n°4 / 4

CV2 (Rayan Kowalski)
Score : 0.5426
Bloc décisif : n°1 / 1

CV12 (Pierre Gautier)
Score : 0.5314
Bloc décisif : n°2 / 2

CV17 (Guillaume Faure)
Score : 0.5283
Bloc décisif : n°1 / 2

CV20 (Isabelle Dumont)
Score : 0.4949
Bloc décisif : n°1 / 1

CV5 (Tariq Mansouri)
Score : 0.4832
Bloc décisif : n°1 / 2

CV18 (Alexia Mercier)
Score : 0.4777
Bloc décisif : n°2 / 3

=====

MEILLEUR CANDIDAT : CV3 (James Fletcher)
Score final : 0.6979

Tous les scores par bloc :
Bloc 0 : 0.5507
Bloc 1 : 0.4008
Bloc 2 : 0.6979 ← meilleur
Bloc 3 : 0.2178

Contenu du bloc décisif (bloc 2) :

Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023. Research Intern, Google Brain London, 2019. Fine-tuning BERT models for question answering. Experimenting with RAG architectures. Python expert, HuggingFace, sentence-transformers, spaCy, PyTorch, FAISS, cosine similarity, embeddings, LLMs, RAG, FastAPI, Docker, Git, GitHub Actions.

[35]: ('CV3 (James Fletcher)', 0.6979)

[]:

7 Stratégie 5 : Fusionner toutes les stratégies précédantes

```
[36]: def chunk_fusion(strategies_resultats):  
        """Fusionne les scores de toutes les stratégies avec pondération_  
        ↪automatique."""
```

```

# 1. Poids automatiques basés sur le max score de chaque stratégie
meta_scores = {
    strat: max(cv["score"] for cv in resultats)
    for strat, resultats in strategies_resultats.items()
}
total = sum(meta_scores.values())
weights = {k: v / total for k, v in meta_scores.items()}

# 2. Construire all_cv_scores
all_cv_scores = {}
for strategy_name, resultats in strategies_resultats.items():
    for cv in resultats:
        nom = cv["nom"]
        if nom not in all_cv_scores:
            all_cv_scores[nom] = {}
        all_cv_scores[nom][strategy_name] = cv["score"]

# 3. Score fusion pondéré
fusion_scores = {
    nom: sum(scores.get(strat, 0) * w for strat, w in weights.items())
    for nom, scores in all_cv_scores.items()
}

# 4. Classement
classement_final = sorted(fusion_scores.items(), key=lambda x: x[1],
↪reverse=True)

print("=" * 60)
print("      CLASSEMENT FINAL (chunk_fusion - fusion pondérée)")
print("=" * 60)
for i, (nom, score) in enumerate(classement_final, 1):
    barre = " " * int(score * 30)
    print(f"\n{i}. {nom}")
    print(f"    Score fusion : {score:.4f}  {barre}")
    for strat, w in weights.items():
        score_strat = all_cv_scores[nom].get(strat, 0)
        print(f"      [{strat}] {score_strat:.4f}  (poids {w:.2f})")
print("\n" + "=" * 60)

# 5. Détail du meilleur candidat
meilleur_nom, meilleur_score = classement_final[0]
print(f"\n MEILLEUR CANDIDAT : {meilleur_nom}")
print(f" Score fusion : {meilleur_score:.4f}")
print(f"\n Blocs décisifs par stratégie :")
print("-" * 60)
for strat, resultats in strategies_resultats.items():

```

```

    for cv in resultats:
        if cv["nom"] == meilleur_nom:
            print(f"\n [{strat}] bloc n°{cv['meilleur_idx'] + 1} /_
↪{cv['nb_blocs']} :")
            print(f" {cv['meilleur_bloc']}")
        print("-" * 60)

    return classement_final

```

```

[37]: strategies_resultats = {
    "chunk_by_words":      resultats_chunk_by_words,
    "chunk_by_sentences":  resultats_chunk_by_sentences,
    "chunk_semantic":      resultats_chunk_semantic,
    "chunk_by_newlines":   resultats_chunk_by_newlines,
}

classement_final = chunk_fusion(strategies_resultats)

```

```

=====
          CLASSEMENT FINAL (chunk_fusion - fusion pondérée)
=====

```

1. CV3 (James Fletcher)
 - Score fusion : 0.6871
 - [chunk_by_words] 0.7233 (poids 0.25)
 - [chunk_by_sentences] 0.5991 (poids 0.25)
 - [chunk_semantic] 0.6979 (poids 0.25)
 - [chunk_by_newlines] 0.7255 (poids 0.25)
2. CV6 (Maxime Berthelot)
 - Score fusion : 0.6847
 - [chunk_by_words] 0.7182 (poids 0.25)
 - [chunk_by_sentences] 0.6523 (poids 0.25)
 - [chunk_semantic] 0.6495 (poids 0.25)
 - [chunk_by_newlines] 0.7163 (poids 0.25)
3. CV8 (Fatou Diallo)
 - Score fusion : 0.6787
 - [chunk_by_words] 0.6787 (poids 0.25)
 - [chunk_by_sentences] 0.6787 (poids 0.25)
 - [chunk_semantic] 0.6787 (poids 0.25)
 - [chunk_by_newlines] 0.6787 (poids 0.25)
4. CV4 (Camille Nguyen)
 - Score fusion : 0.6699
 - [chunk_by_words] 0.6810 (poids 0.25)
 - [chunk_by_sentences] 0.7012 (poids 0.25)

- [chunk_semantic] 0.6378 (poids 0.25)
[chunk_by_newlines] 0.6595 (poids 0.25)
5. CV1 (Sophie Marchand)
Score fusion : 0.6658
[chunk_by_words] 0.6681 (poids 0.25)
[chunk_by_sentences] 0.6387 (poids 0.25)
[chunk_semantic] 0.6387 (poids 0.25)
[chunk_by_newlines] 0.7158 (poids 0.25)
6. CV14 (Yuna Park)
Score fusion : 0.6565
[chunk_by_words] 0.6525 (poids 0.25)
[chunk_by_sentences] 0.6936 (poids 0.25)
[chunk_semantic] 0.6293 (poids 0.25)
[chunk_by_newlines] 0.6508 (poids 0.25)
7. CV10 (Amandine Lefebvre)
Score fusion : 0.6540
[chunk_by_words] 0.7071 (poids 0.25)
[chunk_by_sentences] 0.5269 (poids 0.25)
[chunk_semantic] 0.6822 (poids 0.25)
[chunk_by_newlines] 0.6966 (poids 0.25)
8. CV11 (Nina Castillo)
Score fusion : 0.6485
[chunk_by_words] 0.6446 (poids 0.25)
[chunk_by_sentences] 0.6427 (poids 0.25)
[chunk_semantic] 0.6427 (poids 0.25)
[chunk_by_newlines] 0.6635 (poids 0.25)
9. CV15 (Mehdi Tazi)
Score fusion : 0.6461
[chunk_by_words] 0.6461 (poids 0.25)
[chunk_by_sentences] 0.6461 (poids 0.25)
[chunk_semantic] 0.6461 (poids 0.25)
[chunk_by_newlines] 0.6461 (poids 0.25)
10. CV9 (Théo Blanchard)
Score fusion : 0.6413
[chunk_by_words] 0.6803 (poids 0.25)
[chunk_by_sentences] 0.6450 (poids 0.25)
[chunk_semantic] 0.6450 (poids 0.25)
[chunk_by_newlines] 0.5952 (poids 0.25)
11. CV7 (Lorenzo Ricci)
Score fusion : 0.6243
[chunk_by_words] 0.7089 (poids 0.25)

- [chunk_by_sentences] 0.6174 (poids 0.25)
[chunk_semantic] 0.5847 (poids 0.25)
[chunk_by_newlines] 0.5847 (poids 0.25)
12. CV13 (Sébastien Morin)
Score fusion : 0.5931
[chunk_by_words] 0.5935 (poids 0.25)
[chunk_by_sentences] 0.5929 (poids 0.25)
[chunk_semantic] 0.5929 (poids 0.25)
[chunk_by_newlines] 0.5929 (poids 0.25)
13. CV16 (Lucie Renard)
Score fusion : 0.5693
[chunk_by_words] 0.6093 (poids 0.25)
[chunk_by_sentences] 0.5662 (poids 0.25)
[chunk_semantic] 0.5506 (poids 0.25)
[chunk_by_newlines] 0.5506 (poids 0.25)
14. CV5 (Tariq Mansouri)
Score fusion : 0.5642
[chunk_by_words] 0.6497 (poids 0.25)
[chunk_by_sentences] 0.4832 (poids 0.25)
[chunk_semantic] 0.4832 (poids 0.25)
[chunk_by_newlines] 0.6352 (poids 0.25)
15. CV12 (Pierre Gautier)
Score fusion : 0.5632
[chunk_by_words] 0.6907 (poids 0.25)
[chunk_by_sentences] 0.5033 (poids 0.25)
[chunk_semantic] 0.5314 (poids 0.25)
[chunk_by_newlines] 0.5247 (poids 0.25)
16. CV2 (Rayan Kowalski)
Score fusion : 0.5614
[chunk_by_words] 0.6165 (poids 0.25)
[chunk_by_sentences] 0.5426 (poids 0.25)
[chunk_semantic] 0.5426 (poids 0.25)
[chunk_by_newlines] 0.5426 (poids 0.25)
17. CV17 (Guillaume Faure)
Score fusion : 0.5353
[chunk_by_words] 0.5855 (poids 0.25)
[chunk_by_sentences] 0.5283 (poids 0.25)
[chunk_semantic] 0.5283 (poids 0.25)
[chunk_by_newlines] 0.4987 (poids 0.25)
18. CV18 (Alexia Mercier)
Score fusion : 0.5063

[chunk_by_words] 0.4861 (poids 0.25)
[chunk_by_sentences] 0.5682 (poids 0.25)
[chunk_semantic] 0.4777 (poids 0.25)
[chunk_by_newlines] 0.4940 (poids 0.25)

19. CV19 (Raphaël Cordier)

Score fusion : 0.5057
[chunk_by_words] 0.5214 (poids 0.25)
[chunk_by_sentences] 0.4704 (poids 0.25)
[chunk_semantic] 0.5438 (poids 0.25)
[chunk_by_newlines] 0.4876 (poids 0.25)

20. CV20 (Isabelle Dumont)

Score fusion : 0.5038
[chunk_by_words] 0.5301 (poids 0.25)
[chunk_by_sentences] 0.4949 (poids 0.25)
[chunk_semantic] 0.4949 (poids 0.25)
[chunk_by_newlines] 0.4949 (poids 0.25)

=====

MEILLEUR CANDIDAT : CV3 (James Fletcher)
Score fusion : 0.6871

Blocs décisifs par stratégie :

[chunk_by_words] bloc n°2 / 3 :
for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023. Research Intern, Google Brain London, 2019. Fine-tuning BERT models for question answering. Experimenting with RAG architectures. SKILLS Python expert, HuggingFace, sentence-transformers, spaCy, PyTorch, FAISS, cosine similarity, embeddings, LLMs, RAG, FastAPI, Docker, Git, GitHub Actions. LANGUAGES English native, French C1, German

[chunk_by_sentences] bloc n°1 / 2 :
James Fletcher | Research Scientist - NLP | james.fletcher@email.fr | Grenoble
| 06 33 44 55 66

EDUCATION

PhD Natural Language Processing, University of Edinburgh, 2020. Thesis: Contextual Embeddings for Cross-lingual Semantic Search. MSc Computer Science, University of Edinburgh, 2016. BSc Mathematics, University of Bristol, 2014.

EXPERIENCE

Research Scientist, Naver Labs Europe Grenoble, 2020 to present. Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023.

[chunk_semantic] bloc n°3 / 4 :

Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023. Research Intern, Google Brain London, 2019. Fine-tuning BERT models for question answering. Experimenting with RAG architectures. Python expert, HuggingFace, sentence-transformers, spaCy, PyTorch, FAISS, cosine similarity, embeddings, LLMs, RAG, FastAPI, Docker, Git, GitHub Actions.

[chunk_by_newlines] bloc n°5 / 10 :

Research Scientist, Naver Labs Europe Grenoble, 2020 to present. Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023.

```
[38]: # Conversion en format compatible avec calculer_metriques
resultats_chunk_fusion = [
    {
        "nom":          nom,
        "score":         score,
        "meilleur_idx":  0,
        "meilleur_bloc": "",
        "nb_blocs":      1,
        "scores_blocs":  [score],
    }
    for nom, score in classement_final
]
```

```
[ ]:
```

```
[ ]:
```

```
[39]: def chunk_adaptatif(texte, model=None, max_words=80, seuil_similarite=None):
```

```

# 0. Prétraitement
texte = pretraiter_cv(texte)

# 2. Chunking hiérarchique avec préservation de l'ordre
segments = []
ordre_global = 0

a_traiter_n2, a_traiter_n3, a_traiter_n4 = [], [], []

# Niveau 1 : paragraphes
for bloc in texte.strip().split("\n\n"):
    a_traiter_n2.append((ordre_global, bloc))
    ordre_global += 1

# Fonction utilitaire : nombre de mots
def nb_mots(txt):
    return len(txt.split())

# Fonction pour découper et ajouter en respectant l'ordre
def traiter_niveau(blocs, max_words):
    result = []
    a_traiter_suivant = []

    for idx, bloc in blocs:
        if nb_mots(bloc) <= max_words:
            result.append((idx, bloc.strip()))
        else:
            a_traiter_suivant.append((idx, bloc))
    return result, a_traiter_suivant

# Niveau 2 : lignes
segments_n2, a_traiter_n3 = traiter_niveau([
    (idx, "\n".join(bloc.split("\n"))) for idx, bloc in a_traiter_n2
], max_words)

# Niveau 3 : phrases
segments_n3, a_traiter_n4 = traiter_niveau([
    (idx, phrase)
    for idx, bloc in a_traiter_n3
    for phrase in re.split(r'(?<=[.!?;:])\s+', bloc)
], max_words)

# Niveau 4 : mots (découpage forcé)
segments_n4 = []
for idx, bloc in a_traiter_n4:
    courant = ""
    for mot in bloc.split():

```

```

        if nb_mots(courant) + 1 <= max_words:
            courant += (" " if courant else "") + mot
        else:
            segments_n4.append((idx, courant.strip()))
            courant = mot
    if courant:
        segments_n4.append((idx, courant.strip()))

# Combinaison
segments = segments_n2 + segments_n3 + segments_n4

# Tri
segments.sort(key=lambda x: x[0])

blocs = [texte_s for _, texte_s in segments if len(texte_s) > 0]

# 3. Embeddings
embeddings = model.encode(blocs, show_progress_bar=False)

# 4. Similarités cosinus
sims = [
    cosine_similarity([embeddings[i - 1]], [embeddings[i]])[0][0]
    for i in range(1, len(embeddings))
]

# 5. Seuil adaptatif
if len(sims) == 0:
    seuil = 0.5 # valeur par défaut
else:
    if seuil_similarite is None:
        seuil = float(np.median(sims) + 0.5 * np.std(sims))
        print(seuil)
        seuil = max(0.2, min(seuil, 0.85))
    else:
        seuil = float(seuil_similarite)

# 6. Fusion sémantique bornée (EN MOTS)
blocs_fusionnes = []
groupe_courant = [blocs[0]]
taille_courante = nb_mots(blocs[0])

for i, sim in enumerate(sims):
    prochain = blocs[i + 1]
    taille_fusionnee = taille_courante + nb_mots(prochain)

    if sim >= seuil and taille_fusionnee <= max_words:
        groupe_courant.append(prochain)

```

```

        taille_courante = taille_fusionnee
    else:
        blocs_fusionnees.append(" ".join(groupe_courant))
        groupe_courant = [prochain]
        taille_courante = nb_mots(prochain)

    blocs_fusionnees.append(" ".join(groupe_courant))

    return blocs_fusionnees, embeddings, model

```

```

[40]: cvs_chunk_adaptatif = {nom: chunk_adaptatif(load_file(chemin), model)[0] for
    ↪ nom, chemin in CV_PATHS.items()}

```

```

0.5356923937797546
0.5695083141326904
0.2915545701980591
0.43948182463645935
0.5380480289459229
0.3791921138763428
0.19172176718711853
0.6058960556983948
0.36611008644104004
0.4090186655521393
0.3374900817871094
0.37471622228622437
0.43348345160484314
0.5064772963523865
0.4362652897834778
0.6000242233276367
0.4571889042854309
0.3846827745437622
0.5000300407409668

```

```

[41]: for i, (nom, blocs) in enumerate(cvs_chunk_adaptatif.items()):
    print(f"{nom} => {len(blocs)}")

```

```

CV1 (Sophie Marchand) => 6
CV2 (Rayan Kowalski) => 2
CV3 (James Fletcher) => 7
CV4 (Camille Nguyen) => 6
CV5 (Tariq Mansouri) => 4
CV6 (Maxime Berthelot) => 11
CV7 (Lorenzo Ricci) => 8
CV8 (Fatou Diallo) => 2
CV9 (Théo Blanchard) => 7
CV10 (Amandine Lefebvre) => 5
CV11 (Nina Castillo) => 6

```

```

CV12 (Pierre Gautier) => 4
CV13 (Sébastien Morin) => 2
CV14 (Yuna Park) => 5
CV15 (Mehdi Tazi) => 1
CV16 (Lucie Renard) => 7
CV17 (Guillaume Faure) => 5
CV18 (Alexia Mercier) => 4
CV19 (Raphaël Cordier) => 5
CV20 (Isabelle Dumont) => 2

```

```
[42]: resultats_chunk_adaptatif = extract_results(embedding_offre,
↪cvcs_chunk_adaptatif)
```

```
[43]: classement(resultats_chunk_adaptatif, "chunk adaptatif")
```

```

=====
          CLASSEMENT DES CANDIDATS (avec chunking : chunk adaptatif)
=====

```

```

CV3 (James Fletcher)
  Score : 0.7275
  Bloc décisif : n°5 / 7

```

```

CV1 (Sophie Marchand)
  Score : 0.7158
  Bloc décisif : n°3 / 6

```

```

CV10 (Amandine Lefebvre)
  Score : 0.6966
  Bloc décisif : n°3 / 5

```

```

CV9 (Théo Blanchard)
  Score : 0.6803
  Bloc décisif : n°7 / 7

```

```

CV8 (Fatou Diallo)
  Score : 0.6787
  Bloc décisif : n°1 / 2

```

```

CV4 (Camille Nguyen)
  Score : 0.6595
  Bloc décisif : n°3 / 6

```

```

CV14 (Yuna Park)
  Score : 0.6492
  Bloc décisif : n°3 / 5

```

```

CV15 (Mehdi Tazi)

```

Score : 0.6461
Bloc décisif : n°1 / 1

CV5 (Tariq Mansouri)
Score : 0.6352
Bloc décisif : n°3 / 4

CV19 (Raphaël Cordier)
Score : 0.6296
Bloc décisif : n°4 / 5

CV13 (Sébastien Morin)
Score : 0.5935
Bloc décisif : n°1 / 2

CV7 (Lorenzo Ricci)
Score : 0.5847
Bloc décisif : n°4 / 8

CV2 (Rayan Kowalski)
Score : 0.5846
Bloc décisif : n°2 / 2

CV6 (Maxime Berthelot)
Score : 0.5845
Bloc décisif : n°9 / 11

CV11 (Nina Castillo)
Score : 0.5588
Bloc décisif : n°6 / 6

CV20 (Isabelle Dumont)
Score : 0.5381
Bloc décisif : n°2 / 2

CV16 (Lucie Renard)
Score : 0.5303
Bloc décisif : n°2 / 7

CV12 (Pierre Gautier)
Score : 0.5247
Bloc décisif : n°4 / 4

CV18 (Alexia Mercier)
Score : 0.5241
Bloc décisif : n°3 / 4

CV17 (Guillaume Faure)

Score : 0.4987
Bloc décisif : n°3 / 5

=====

MEILLEUR CANDIDAT : CV3 (James Fletcher)
Score final : 0.7275

Tous les scores par bloc :
Bloc 0 : 0.2772
Bloc 1 : 0.2854
Bloc 2 : 0.5294
Bloc 3 : 0.2995
Bloc 4 : 0.7275 ← meilleur
Bloc 5 : 0.2680
Bloc 6 : 0.6400

Contenu du bloc décisif (bloc 4) :

Research Scientist, Naver Labs Europe Grenoble, 2020 to present. Developing state-of-the-art NLP models for multilingual semantic search. Building pipelines with Python, HuggingFace Transformers and sentence-transformers. Computing cosine similarity at scale using FAISS. Deploying inference APIs with FastAPI and Docker. Contributing to open-source NLP projects on GitHub. Publications: ACL 2021, EMNLP 2022, NAACL 2023. Research Intern, Google Brain London, 2019. Fine-tuning BERT models for question answering. Experimenting with RAG architectures.

[43]: ('CV3 (James Fletcher)', 0.7275)

[]:

[]:

```
[45]: from scipy.stats import spearmanr
from sklearn.metrics import ndcg_score
import numpy as np

# Classement attendu (rang 1 = meilleur)
classement_attendu = {
    "CV3 (James Fletcher)": 1,
    "CV10 (Amandine Lefebvre)": 2,
    "CV1 (Sophie Marchand)": 3,
    "CV2 (Rayan Kowalski)": 4,
    "CV15 (Mehdi Tazi)": 5,
    "CV4 (Camille Nguyen)": 6,
```

```

"CV9 (Théo Blanchard)": 7,
"CV5 (Tariq Mansouri)": 8,
"CV14 (Yuna Park)": 9,
"CV6 (Maxime Berthelot)": 10,
"CV12 (Pierre Gautier)": 11,
"CV8 (Fatou Diallo)": 12,
"CV7 (Lorenzo Ricci)": 13,
"CV13 (Sébastien Morin)": 14,
"CV11 (Nina Castillo)": 15,
"CV19 (Raphaël Cordier)": 16,
"CV17 (Guillaume Faure)": 17,
"CV16 (Lucie Renard)": 18,
"CV18 (Alexia Mercier)": 19,
"CV20 (Isabelle Dumont)": 20,
}

def calculer_metriques(resultats, classement_attendu, strategy_name):
    """
    Calcule NDCG, Spearman, Precision@3, Precision@5, MRR
    """
    # Classement obtenu par l'algo (rang 1 = meilleur)
    resultats_tries = sorted(resultats, key=lambda x: x["score"], reverse=True)
    classement_obtenu = {r["nom"]: i + 1 for i, r in enumerate(resultats_tries)}

    noms = list(classement_attendu.keys())

    # Scores relatifs (20 = meilleur, 1 = moins bon)
    scores_attendus = [20 - classement_attendu[n] + 1 for n in noms]
    scores_obtenus = [20 - classement_obtenu.get(n, 20) + 1 for n in noms]

    # NDCG
    ndcg = ndcg_score([scores_attendus], [scores_obtenus])

    # Spearman
    rang_attendu = [classement_attendu[n] for n in noms]
    rang_obtenu = [classement_obtenu.get(n, 20) for n in noms]
    spearman, _ = spearmanr(rang_attendu, rang_obtenu)

    # Precision@3
    top3_attendu = set(n for n, r in classement_attendu.items() if r <= 3)
    top3_obtenu = set(n for n, r in classement_obtenu.items() if r <= 3)
    precision_3 = len(top3_attendu & top3_obtenu) / 3

    # Precision@5
    top5_attendu = set(n for n, r in classement_attendu.items() if r <= 5)
    top5_obtenu = set(n for n, r in classement_obtenu.items() if r <= 5)
    precision_5 = len(top5_attendu & top5_obtenu) / 5

```

```

# MRR
meilleur_attendu = min(classement_attendu, key=classement_attendu.get)
rang_meilleur = classement_obtenu.get(meilleur_attendu, 20)
mrr = 1 / rang_meilleur

metriques = {
    "stratégie": strategy_name,
    "NDCG": round(ndcg, 4),
    "Spearman": round(spearman, 4),
    "Precision@3": round(precision_3, 4),
    "Precision@5": round(precision_5, 4),
    "MRR": round(mrr, 4),
}

return metriques

def afficher_metriques(toutes_metriques):
    """Affiche un tableau comparatif de toutes les stratégies."""
    print("\n" + "=" * 75)
    print("    COMPARAISON DES STRATÉGIES DE CHUNKING")
    print("=" * 75)
    print(f"{'Stratégie':<25} {'NDCG':>8} {'Spearman':>10} {'P@3':>6} {'P@5':>6} {'MRR':>8}")
    print("-" * 75)
    for m in toutes_metriques:
        print(f"{'m[stratégie]':<25} {'m[NDCG]':>8.4f} {'m[Spearman]':>10.4f} "
              f"{'m[Precision@3]':>6.2f} {'m[Precision@5]':>6.2f} {'m[MRR]':>8.4f}")
    print("=" * 75)

    # Meilleure stratégie par métrique
    print("\n MEILLEURE STRATÉGIE PAR MÉTRIQUE :")
    for metrique in ["NDCG", "Spearman", "Precision@3", "Precision@5", "MRR"]:
        meilleure = max(toutes_metriques, key=lambda x: x[metrique])
        print(f"    {metrique:<12} → {meilleure['stratégie']}")
    print(f"    ({meilleure[metrique]:.4f})")
    print("=" * 75)

# Appel
toutes_metriques = [
    calculer_metriques(resultats_chunk_by_newlines, classement_attendu,
    ↪ "chunk_by_newlines"),
    calculer_metriques(resultats_chunk_by_words, classement_attendu,
    ↪ "chunk_by_words"),

```

```

    calculer_metriques(resultats_chunk_by_sentences,classement_attendu,↵
↵"chunk_by_sentences"),
    calculer_metriques(resultats_chunk_semantic,  classement_attendu,↵
↵"chunk_semantic"),
    calculer_metriques(resultats_chunk_adaptatif,  classement_attendu,↵
↵"chunk_adaptatif"),
    calculer_metriques(resultats_chunk_fusion,      classement_attendu,↵
↵"chunk_fusion"),
]

afficher_metriques(toutes_metriques)

```

=====					
COMPARAISON DES STRATÉGIES DE CHUNKING					
=====					
Stratégie	NDCG	Spearman	P@3	P@5	MRR

chunk_by_newlines	0.9516	0.6827	0.67	0.60	1.0000
chunk_by_words	0.9361	0.6647	0.33	0.40	1.0000
chunk_by_sentences	0.8808	0.2947	0.00	0.20	0.1000
chunk_semantic	0.9554	0.6030	0.67	0.60	1.0000
chunk_adaptatif	0.9789	0.7805	1.00	0.60	1.0000
chunk_fusion	0.9369	0.6241	0.33	0.40	1.0000
=====					

MEILLEURE STRATÉGIE PAR MÉTRIQUE :

NDCG → chunk_adaptatif (0.9789)
 Spearman → chunk_adaptatif (0.7805)
 Precision@3 → chunk_adaptatif (1.0000)
 Precision@5 → chunk_by_newlines (0.6000)
 MRR → chunk_by_newlines (1.0000)

=====

[]:

[]:

[]: